Find an origin IP behind a CDN (Akamai, Cloudflare, etc.).

AUTHOR: MUSTAFA-ALMOHSEN

1) Certificate Transparency (fast, high-value)
2) Passive subdomain enumeration
3) Passive DNS / Historical DNS / Search engines
4) ASN → CIDR correlation
5) (Non-invasive) SNI certificate check on suspected IPs
7) Reverse DNS sweep of CIDR (use carefully / only if allowed)
One-Page-Cheat-Sheet:
🔑 Step-by-step (practical, bug bounty–safe)
1. **Understand the basics**
2. **Collect subdomains**
3. **Check DNS records**
4. **Verify with TLS SNI trick**
6. **Repeat for all subdomains**
⚡ Example from your case

Recon workflow for **when we hit** `403` **on an origin IP** `such in step 5 above.`

B — Enumerate accessible resources (some may bypass root 403)
C — Probe for virtual-host routing / host header differences
E — Check allowed methods & CORS
G — Check other ports / services (careful, only if in-scope)

A **compact, safe Bash script** that automates:

1) Certificate Transparency (fast, high-value)

Why: certs often contain internal names or alternate domains.

Command:

```
curl -s "https://crt.sh/?q=%25example.com&output=json" | jq -r '.[].name_value' | sort -u
```

What to look for: `origin.example.net`, `internal.example.com`, other unexpected SANs.

Example expected output:

```
dev.example.com
origin.example.net
internal.example.com
```

2) Passive subdomain enumeration

Why: reveals subdomains that might hint origin names.

Commands:

```
amass enum -passive -d example.com -o amass_passive.txt
subfinder -d example.com -o subfinder.txt
cat amass_passive.txt subfinder.txt | sort -u
```

3) Passive DNS / Historical DNS / Search engines

Use SecurityTrails, VirusTotal, PassiveTotal, Censys, Shodan (web UI or API) to see historic A records or PTRs.

Search `example.com` in Censys/Shodan for IPs and certificate links.

What to extract: any IPs previously associated with internal names or certs.

4) ASN → CIDR correlation

Why: origin often sits in same ASN/netblock as other corporate services.

Find ASN for an IP you found (or for company names):

```
whois 198.51.100.23
# OR
whois -h whois.cymru.com " -v 198.51.100.23"

# asn
┌──(kali㉿kali)-[~]
└─$ whois -h whois.cymru.com "-v AS16509"
Warning: RIPE flags used with a traditional server.
AS      | CC | Registry | Allocated  | AS Name
16509   | US | arin     | 2000-05-04 | AMAZON-02, US
```

Record the CIDR (e.g. `198.51.100.0/24`) and ASN (e.g. `AS12345`).

5) (Non-invasive) SNI certificate check on suspected IPs

Why: the origin may present a different certificate when you connect directly to the origin IP with SNI.

Command:

```
echo | openssl s_client -connect 198.51.100.23:443 -servernam
e dev.example.com 2>/dev/null | openssl x509 -noout -text | s
ed -n '1,120p'
```

What to look for: SANs / CN that match internal names (`origin.example.net`) — strong indicator.

Example snippet you might see in cert:

```
X509v3 Subject Alternative Name:
    DNS:origin.example.net, DNS:api.example.com
```

7) Reverse DNS sweep of CIDR (use carefully / only if allowed)

Why: PTRs sometimes contain hostnames revealing origin.

Command (use with permission):

```
prips 198.51.100.0/24 | hakrevdns -r 8.8.8.8 > revdns.txt
grep -i "example" revdns.txt
```

If PTRs show `origin.example.net` → another confirming data point.

One-Page-Cheat-Sheet:

```
# 1) CT lookup
curl -s "https://crt.sh/?q=%25example.com&output=json" | jq -
r '.[].name_value' | sort -u

# 2) Passive subdomain collection
amass enum -passive -d example.com -o amass_passive.txt
subfinder -d example.com -o subfinder.txt

# 3) ASN for candidate IP
whois 198.51.100.23
# or
whois -h whois.cymru.com " -v 198.51.100.23"

# 4) SNI cert check on suspected IP
echo | openssl s_client -connect 198.51.100.23:443 -servernam
e dev.example.com 2>/dev/null | openssl x509 -noout -text | s
ed -n '1,120p'

# 5) HTTP headers (HEAD)
curl -I -H "Host: dev.example.com" http://198.51.100.23 --max
-time 10

# 6) Reverse DNS of CIDR (only if allowed)
```

```
prips 198.51.100.0/24 | hakrevdns -r 8.8.8.8 > revdns.txt
grep -i "example" revdns.txt
```

🔑 Step-by-step (practical, bug bounty–safe)

1. Understand the basics

- **ASN (Autonomous System Number):** tells you *who owns the IP ranges*. Example: AS13335 = Cloudflare ~ AS16509 = Amazon AWS.
- **CIDR (e.g. `205.251.249.0/24`):** that's just a subnet block = a group of IPs inside an ASN.

So if you see:

```
***.com → AS13335 (Cloudflare)  ---> I should Ignore it cuz b
ehind cloudflar

sapi-live.lh.us-west-2.prd.***me.us → AS16509 (Amazon) --> Pr
omising go for it!
```

That means *****.com is behind Cloudflare**, and **the API is directly on AWS**.

2. Collect subdomains

✅ Using crt.sh:

```
curl -s "https://crt.sh/?q=***.com&output=json" | jq -r '.[].
name_value' | sort -u
```

```
┌──(kali㉿kali)-[~]
└─$ curl -s "https://crt.sh/?q=***.com&output=json" | jq -r
```

```
'.[].name_value' | sort -u
blog.***.com
click.mailer.***.com
*.clinic.***.com
cloud.mailer.***.com
healthaid.***.com
image.mailer.***.com
*.***.com
***.com
mailer.***.com
maintenance.***.com
maintenance-page.***.com
*.pims.***.com
*.pims.staging.***.com
primarycare-chat.***.com
primarycare-go.***.com
primarycare.***.com
support.***.com
test-web.***.com
try.***.com
url2031.***.com
view.mailer.***.com
www.blog.***.com
www.primarycare-go.***.com
```

3. Check DNS records

Run `dig`:

```
dig +short test-web.***.com
dig cname test-web.***.com
```

- **If it resolves to Cloudflare IP ranges (AS13335, AS209242, etc.) → protected.**
- **If it resolves to AWS/Azure/other hosting ASN → potential origin IP.**

Example from your output:

```
test-web.***.com → thirdparty.bnc.lt → 52.11.118.109 (AWS, AS
16509)
```

This one is **not behind Cloudflare** → candidate origin.

4. Verify with TLS SNI trick

Use your OpenSSL method, but smarter:

```
echo | openssl s_client -connect 34.197.112.110:443 -serverna
me test-web.***.com 2>/dev/null | openssl x509 -noout -subjec
t -issuer -ext subjectAltName
```

- If the cert matches `test-web.***.com` → that IP is serving the site directly = origin.
- If not (default cert, mismatch, or CDN wildcard) → discard.

5. **Cross-check via HTTP**

```
curl -s -k --resolve test-web.***.com:443:34.197.112.110 http
s://test-web.***.com -I
```

- **If you get a valid 200/302/403 with proper headers → bingo (real backend).**

- **If you get default AWS/CloudFront error → still behind CDN.**

6. Repeat for all subdomains

- Focus on weird ones like:
  - `test-*.domain.com`
  - `staging.domain.com`
  - `dev.domain.com`
  - `mailer.domain.com`
  - `api.domain.com`
- These are often **forgotten and exposed** directly on AWS/GCP.

⚡ Example from your case

- `***.com` → Cloudflare → ignore.
- `url2031.***.com` → points to AWS IP `52.11.118.109`.
  - TLS cert check + curl resolve = confirm if it's a **real backend** or just a third-party service.

So instead of brute-forcing the *entire* CloudFront CIDR (which is huge and noisy), the better way is:

- **Enumerate subdomains.**
- **Check which ones resolve to non-Cloudflare IPs.**
- **Verify them via SNI/TLS + HTTP.**

Recon workflow for when we hit `403` on an origin IP `such in step 5 above.`

- in Example above we already have `34.197.112.110` serving `test-web.***.com` and it returned `403`.

1. **Save everything**: raw `openssl` output, `curl -I --resolve` output, `dig +short`, `whois` of IP.

```
echo | openssl s_client -connect 34.197.112.110:443 -serverna
me test-web.***.com 2>/dev/null | openssl x509 -noout -text >
cert.txt
curl -I -k --resolve test-web.***.com:443:34.197.112.110 http
s://test-web.***.com -o headers.txt
dig +short test-web.***.com > digout.txt
whois 34.197.112.110 > whois_ip.txt
```

2. **Check Certificate Transparency & history** (already did via crt.sh)

B — Enumerate accessible resources (some may bypass root 403)

Try these (non-aggressive) endpoints — staging/test often exposes:

```
# common files & discovery endpoints
endpoints=(/robots.txt /sitemap.xml /.env /.git/ /admin /logi
n /status /health /healthz /.well-known/security.txt /config)
for p in "${endpoints[@]}"; do
  curl -s -k --resolve test-web.***.com:443:34.197.112.110 "h
ttps://test-web.***.com${p}" -I | sed -n '1,4p'
done
```

C — Probe for virtual-host routing / host header differences

Sometimes the origin responds to a different Host header (vhost) or another subdomain on same IP is unprotected.

1. **Find other vhosts on the IP** (HTTP virtual host scan; passive):

```
# using a small wordlist of likely hostnames (e.g., from crt.
sh results)
```

```
cat candidates_hosts.txt | while read h; do
  curl -s -k --resolve $h:443:34.197.112.110 https://$h -I |
sed -n '1,3p'
done
```

2.**Test alternate Host header** (sometimes admin panels listen on a different host

```
curl -s -k -I -H "Host: admin.test-web.***.com" --resolve adm
in.test-web.***.com:443:34.197.112.110 https://admin.test-we
b.***.com
```

D — Directory brute-force (conservative)

```
ffuf -u https://test-web.***.com/FUZZ -w /usr/share/wordlist
s/dirb/common.txt -H "Host: test-web.***.com" -k -t 20 -p 0.2
--timeout 8 -r --resolve test-web.***.com:443:34.197.112.110
```

E — Check allowed methods & CORS

1. **OPTIONS / Allowed methods**

```
curl -X OPTIONS -s -k -I --resolve test-web.***.com:443:34.19
7.112.110 https://test-web.***.com
```

2. **CORS test** (if any API endpoints discovered)

```
curl -s -k -I -H "Origin: https://evil.com" --resolve test-we
b.***.com:443:34.197.112.110 https://test-web.***abdnd.com/so
mepath
```

G — Check other ports / services (careful, only if in-scope)

A basic port/service check can reveal admin panels (SSH/RDP, management ports).

```
# Quick port scan (top common ports) - non-invasive
nmap -sT -p22,80,443,8080,8443,8000 --reason -Pn 34.197.112.1
10
```

A compact, safe Bash script that automates:

subdomain enum → DNS resolution → ASN lookup → TLS (SNI) cert check → HTTP header check (via --resolve)
It prints only **candidate origin IPs** (IPs that present a cert matching the subdomain or give relevant headers).

- Check the tool

```
```